

KONERU LAKSHMAIAH EDUCATION FOUNDATION**AZIZ NAGAR, HYDERABAD****DEPARTMENT OF ECE****Project Proposal**

1.0	Details of Candidates:	(i) MD Sameer Ahmed (2310040004) (ii) Sayooj S (2310040095)
	Course of Study:	B.TECH/ECE
	Year:	IV
	Semester:	I
	Batch No.:	
2.0	Course Details:	SYSTEM-ON-CHIP (23VLS3404) MIXED SIGNAL IC DESIGN (23VLS3505)
3.0	Name of Supervisor:	Dr. Phalguni Singh
4.0	Proposed Title:	Design and Evaluation of a Configurable AI Acceleration Extension for an Embedded RISC-V Processor on FPGA

August, 2026

5.0 Introduction

5.1 General Introduction

The rapid proliferation of the Internet of Things (IoT) and embedded computing has fundamentally changed the way data is processed. Modern edge devices — sensors, wearables, drones, industrial controllers, and battery-powered medical monitors — are increasingly required to run artificial intelligence (AI) and machine learning (ML) workloads locally rather than streaming data to the cloud. This shift toward edge AI and TinyML is motivated by compelling advantages: reduced network congestion, lower latency, better data privacy, and substantially lower energy consumption than remote data-center processing (Gorde et al., 2025). However, moving intelligent computation to the edge places severe constraints on the underlying hardware. These devices must deliver meaningful throughput and accuracy while operating within power budgets of a few milliwatts to a few hundred milliwatts, and they must fit within tiny silicon or logic budgets.

Conventional embedded processors are poorly matched to this challenge. A typical microcontroller or soft processor is a single-issue, in-order machine that executes instructions sequentially. AI kernels such as matrix multiplication, convolution, and dot products are data-parallel by nature — they consist of millions of multiply–accumulate (MAC) operations that can, in principle, be executed concurrently. On a scalar CPU, each of these operations becomes a long sequence of load, multiply, and add instructions. The result is that such workloads suffer from low instruction throughput, severe memory-bandwidth bottlenecks, and very poor energy efficiency (Tortorella et al., 2022). This is a fundamental architectural mismatch rather than a mere implementation deficiency.

Several classes of solutions have emerged to close this gap. On one end of the spectrum are fully dedicated hardware accelerators such as the Google Tensor Processing Unit (TPU) (Jouppi et al., 2017), Eyeriss (Chen et al., 2017), and a growing family of inference chips. These achieve exceptional throughput and energy efficiency for deep neural network (DNN) inference. On the other end are general-purpose processors that trade efficiency for programmability. Between these extremes lies the approach adopted in this project: extending a general-purpose embedded processor with a custom, application-specific accelerator — an approach known in the literature as an Application-Specific Instruction Set Processor (ASIP) (Rezunov et al., 2025).

The RISC-V instruction set architecture (ISA) has become the preferred foundation for this kind of design (Waterman and Asanović, 2019). RISC-V is an open, freely available, modular ISA that provides a stable base (RV32I) and explicitly reserves opcode space for custom and vendor extensions. Because the ISA is open, and because a large ecosystem of open-source cores exists — PicoRV32, Ibex, VexRiscv, and others — researchers can legally and practically modify the instruction set, the decoder, and the datapath to integrate bespoke hardware units. PicoRV32 (Wolf, 2016) is an especially attractive baseline for a first research project because of its tiny footprint, its simple and well-documented interface, and the fact that it already provides a PCPI (Pico Co-Processor Interface) through which custom instructions can be offloaded to external hardware.

Field-Programmable Gate Arrays (FPGAs) complete the picture. An FPGA provides a reconfigurable fabric built from lookup tables (LUTs), block RAM (BRAM), and embedded DSP slices that can implement multiply–accumulate operations at high speed. FPGAs allow a designer to prototype an entire processor-plus-accelerator system, synthesize it, and measure real timing, area, and power results in a

matter of hours. This makes the FPGA the ideal platform for architectural research: multiple accelerator configurations can be explored and compared without committing to fabrication. The Xilinx Artix-7 XC7A35T on the Digilent Basys 3 board, used in this project, is a representative low-cost embedded FPGA with enough resources for a soft RISC-V core and a moderate-size MAC engine.

This project therefore sits at the intersection of three enabling technologies: an open embedded ISA (RISC-V), a reconfigurable prototyping platform (FPGA), and data-parallel AI kernels. The central idea is to design a configurable AI acceleration extension — a parametrizable MAC-array engine integrated with an embedded RISC-V processor through custom instructions — and to evaluate it quantitatively across several configurations and benchmark workloads. By making the accelerator configurable (number of MAC units, operand widths, and integration approach), the project investigates the area–performance–energy trade-off space that real designers face when building edge-AI systems. The remainder of this report formalizes the problem, states the objectives, defines the scope, presents a literature review, and describes the planned methodology, expected outcomes, budget, and schedule.

5.2 Problem Statement

- (i) Embedded scalar processors such as the PicoRV32 RV32I core execute data-parallel AI workloads matrix multiplication, convolution, FIR filtering, and dot products — as long sequences of individual load, multiply, and add instructions. Because these cores issue only a single instruction per cycle and expose limited memory bandwidth, running AI kernels in software on such cores results in poor execution time, low throughput, and high energy consumption relative to the parallelism inherent in the algorithms. There is a need to accelerate these kernels without abandoning the programmability and low cost of the embedded processor.
- (ii) Existing hardware accelerators are typically fixed-configuration designs that are either tightly specialized to one network topology or deployed as rigid co-processors with limited programmability and little ability to be reconfigured. Off-the-shelf AI IP often cannot be adapted to a given application's accuracy, power, and resource budget, and its integration into an open-source RISC-V core is frequently hampered by a lack of standardized interfaces for custom instruction extension. There is a need for a configurable accelerator whose dimensions (number of MAC units, operand width) can be tuned, and which can be integrated into an embedded RISC-V processor cleanly and evaluated on an FPGA.

5.3 Objectives of the study

The general objective of this study is to design and evaluate a configurable AI acceleration extension for an embedded RISC-V processor on an FPGA, quantifying the performance, area, and energy trade-offs across multiple accelerator configurations and representative AI benchmark workloads

- (i) To design and implement a configurable multiply–accumulate (MAC) based AI accelerator extension and integrate it with an embedded PicoRV32 RISC-V core on a Xilinx Artix-7 FPGA through custom RISC-V instructions, so that data-parallel AI kernels (matrix multiplication, convolution, dot product, FIR, image filters) are offloaded from software into hardware

- (ii) To synthesize and evaluate the integrated system across multiple accelerator configurations (2, 4, and 8 MAC units) and benchmark workloads on the FPGA, comparing execution time, speedup, clock frequency, area utilization, and power consumption against the software baseline, in order to establish the configuration trade-off space.

5.4 Scope of the Project

This project covers the design, implementation, verification, and FPGA-based evaluation of a configurable AI acceleration extension for an embedded RISC-V processor. The hardware scope includes a baseline RV32I soft-core processor (PicoRV32) implemented on the Digilent Basys 3 board (Xilinx Artix-7 XC7A35T), together with a parameterizable accelerator engine built around a configurable array of MAC units, a scratchpad memory, and a controller. The accelerator is attached to the core through custom RISC-V instructions of the form MATMUL, CONV, MAC, DOT, LOAD_TILE, STORE_TILE, and WAIT, which require modification of the decoder, execute stage, and control logic. A full RTL design, written in SystemVerilog, and a verification environment using SystemVerilog testbenches and/or Cocotb are within scope, including randomized and software-vs-hardware differential testing. On the FPGA side, the project covers synthesis, implementation, timing closure, resource-utilization and power reporting, and on-board demonstration. The evaluation scope is a benchmark suite of 4–6 kernels — matrix multiplication, Sobel edge detection, Gaussian blur, FIR filter, dot product, and a tiny CNN — measured in terms of execution cycles, speedup over the software baseline, maximum clock frequency, LUT/BRAM/DSP utilization, and estimated power. A design-space exploration across 2/4/8-MAC configurations and operand widths (INT8/INT16/INT32) is included, since this exploration constitutes the main research contribution.

The following are explicitly out of scope: full system-on-chip integration with peripherals and operating systems; multi-core clusters; ASIC fabrication or tape-out; on-chip training in floating point; analog or in-memory-computing accelerators; and porting of the accelerator to other FPGA families or processor cores. The project is hardware-oriented and is bounded by the resources of the Basys 3 board; all results are reported at the FPGA-prototype level.

5.5 Literature Review

A literature survey is a systematic search of published works to locate relevant information on a specific topic, and it is an essential step in research because it (a) identifies what has already been written, (b) reveals interpretable trends in a research area, (c) aggregates empirical findings to support evidence-based practice, (d) helps generate new frameworks, and (e) uncovers topics that require further investigation. This section reviews the literature that motivates and frames the present work, following the "journey from the base paper" approach in which the claims of each reference are traced back to the earlier works that support them.

The case for edge-AI hardware. The demand for energy-efficient AI hardware is well established. Gorde et al. (2025) provide a comprehensive review of hardware design trends for neural networks, arguing that conventional digital processors are constrained by sequential data processing, memory-bandwidth limitations, and high power consumption, making them suboptimal for edge-based AI. Their survey examines both digital and analog VLSI approaches — in-memory analog computing, current-mode circuits, switched-capacitor techniques, and OTA-based designs — and identifies precision–linearity trade-offs and mixed-signal interface challenges. While their work focuses on analog computing as an

alternative, it documents the same underlying problem that motivates this project: general-purpose digital cores do not deliver the throughput per watt that neural workloads require, and dedicated hardware structures (multiply-accumulate arrays, systolic organizations) are needed. The present project adopts the more mainstream digital-FPGA path, which trades the ultimate energy efficiency of analog/in-memory designs for the programmability and rapid prototyping that an FPGA affords.

Data-center and inference accelerators. At the high end of the accelerator spectrum, Jouppi et al. (2017) presented the Google Tensor Processing Unit, a systolic-array-based accelerator that delivered 15–30× speedup over contemporaneous GPU/CPU baselines for DNN inference, establishing the systolic/MAC-array organization as the de-facto standard for matrix-heavy AI kernels. Chen et al. (2017) proposed Eyeriss, an energy-efficient reconfigurable accelerator for convolutional neural networks built on a row-stationary dataflow that minimizes data movement — a key insight: in AI acceleration, energy is dominated by memory traffic, not arithmetic. Table 5.1 summarizes representative inference accelerators and the arithmetic/datapath choices that this project borrows from.

Accelerator	Reference (Author, Year)	Technology	Arithmetic	Key Result
TPU	Jouppi et al., 2017	28 nm ASIC	INT8	Systolic array, 15–30× speedup for DNN inference
Eyeriss	Chen et al., 2017	65 nm ASIC	INT16	Row-stationary dataflow, energy-efficient CNN inference
EIE	Han et al., 2016	45 nm ASIC	INT8	Sparse compressed-DNN inference, 590 mW
Simba	Chen et al., 2018	16 nm ASIC	INT8	9.1 TOPS/W, scalable multi-chip DNN inference
RedMule	Tortorella et al., 2022	22 nm ASIC	FP16	666 MHz, 31.6 MAC/cycle, 688 GFLOPS/W

Table 5.1: Representative AI accelerators reviewed for this project.

These works confirm two principles adopted here: MAC/systolic arrays for throughput, and attention to data movement for energy.

Tightly coupled accelerators for RISC-V. The base paper for the integration approach is RedMule by Tortorella et al. (2022), a compact FP16 matrix-multiplication engine conceived for tight integration within a cluster of tiny RISC-V cores based on the PULP architecture (Fig. 1). RedMule is a parametric accelerator whose datapath width and number of FMAs can be configured; a 32-FMA instance occupies just 0.07 mm² (14% of an 8-core RISC-V cluster) in 22 nm, runs at up to 666 MHz, and reaches 31.6 MAC/cycle at 98.8% utilization, delivering 4.65× higher energy efficiency and 22× speedup over software execution on 8 RISC-V cores. Two lessons from RedMule are central to this project: (a) the accelerator must be tightly coupled to the core/cluster so that operand delivery does not become the bottleneck, and (b) making the engine parametric enables direct exploration of the design space. The FPGA-based project

generalizes this idea by making the MAC count and operand width the configurable parameters and evaluating them on reconfigurable logic instead of an ASIC.

Interfaces for custom instruction extension. Integrating a custom unit into a RISC-V core requires a well-defined mechanism for offloading instructions. Damian et al. (2022) present SCAIE-V, an open-source scalable interface for ISA extensions for RISC-V processors. Their work systematically compares existing interfaces — PicoRV32's PCPI, and the CV32E40X offloading interface — and notes that PCPI supports neither custom control flow nor memory transactions, which limits the complexity of accelerators that can be attached to PicoRV32 without modification (Table 5.2).

Core	Interface	Multi-cycle instr.	Memory access	Control flow
PicoRV32	PCPI	Yes	No	No
CV32E40X	Offloading	Yes	Limited	No
SCAIE-V	ISAX	Yes	Yes	Yes

Table 5.2: Capabilities of custom-instruction interfaces for RISC-V cores (after Damian et al., 2022).

This comparison directly informs the integration strategy: for the modest accelerator in this project, the PCPI mechanism — or a minimal custom extension of the decoder/execute path — is sufficient, whereas fully decoupled accelerators with memory transactions would require an interface at the level of SCAIE-V.

Automatic custom-instruction design. Because hand-designing custom instructions is labor-intensive, Rezunov et al. (2025) propose CIDRE, an automatic microarchitecture-aware framework that analyzes hot spots in RISC-V applications and generates custom-instruction candidates, integrating them into an nML processor model so that performance and area can be assessed accurately. In benchmarks from Embench and MiBench, CIDRE achieved up to $2.47\times$ speedup with less than 24% area increase. This work confirms that custom instructions for RISC-V yield substantial gains on exactly the kinds of loop-dominated, data-intensive kernels used in this project, and it validates the expected magnitude of speedup for an ASIP-style design.

Research gap. The reviewed literature covers fixed inference accelerators (Jouppi et al., 2017; Chen et al., 2017), tightly coupled parametric engines for multicore clusters on ASIC (Tortorella et al., 2022), and interfaces/automation for RISC-V custom instructions (Damian et al., 2022; Rezunov et al., 2025). What is missing is a systematic, FPGA-based study of a configurable single-core RISC-V accelerator in which the MAC count and operand width are varied and the resulting speedup/area/energy trade-offs are quantified on real FPGA fabric using a simple, widely used baseline core. This project fills that gap.

6.0 Abstract

The growing deployment of AI and machine learning at the edge has created a strong demand for energy-efficient hardware that can run data-parallel workloads such as matrix multiplication, convolution, and dot products within strict power and cost budgets. Conventional embedded processors — including the single-issue, in-order PicoRV32 RISC-V core — execute these workloads as long sequences of scalar instructions, which results in poor execution time, low throughput, and high energy consumption. At the same time, existing accelerators are typically fixed-configuration designs that cannot be tuned to an application's accuracy, power, or resource budget, and their integration into open-source RISC-V cores is often hindered by the lack of standardized extension interfaces. These two problems motivate the need for a configurable AI acceleration extension for an embedded RISC-V processor.

This project proposes the design and FPGA-based evaluation of a configurable multiply-accumulate (MAC) accelerator engine integrated with a PicoRV32 RV32I core through custom RISC-V instructions (MATMUL, CONV, MAC, DOT, LOAD_TILE, STORE_TILE, WAIT). The system is implemented in SystemVerilog on the Xilinx Artix-7 XC7A35T FPGA of the Digilent Basys 3 board. The accelerator parameterizes the number of MAC units (2/4/8) and the operand width (INT8/INT16/INT32), includes a scratchpad and controller, and is verified with SystemVerilog/Cocotb testbenches including differential testing against software references. A benchmark suite of matrix multiplication, Sobel edge detection, Gaussian blur, FIR filter, dot product, and a tiny CNN is used to measure the design across configurations.

The expected outcome is a working accelerated RISC-V system on FPGA demonstrating multi-fold speedups over the software baseline on AI kernels, with detailed trade-off analysis of speedup, clock frequency, LUT/BRAM/DSP utilization, and power across configurations. Anticipated results include 5–20× speedup on the benchmark kernels for the 8-MAC configuration, a modest (10–30%) increase in logic utilization relative to the baseline core, and an explicit characterization of how performance-per-area scales with MAC count — together with a validated design-space methodology that can guide future edge-AI accelerator design.

7.0 Methodology

The project was carried out in five phases. First, the problem was identified through a review of existing systems and user requirements. Next, the system requirements were analyzed, and the appropriate hardware and software tools were selected. A system architecture and flowchart were then designed to illustrate the working process. The proposed solution was implemented using the selected technologies. Finally, the developed system was tested under different operating conditions, and its performance was evaluated based on accuracy, efficiency, and reliability.

1. Problem Identification. The engineering problem was defined through the literature survey: scalar embedded cores are inefficient for data-parallel AI kernels, and existing accelerators lack configurability and clean RISC-V integration. The gap analysis (Section 5.5) established the need for a configurable MAC-based extension to an embedded RISC-V core evaluated on FPGA.

2. Requirement Analysis. Functional requirements include: a working RV32I core on FPGA; a configurable MAC engine; custom instructions for kernel offload; and a benchmark and reporting framework. Technical requirements include the Basys 3 board (Xilinx Artix-7 XC7A35T), Vivado 2025.2 for synthesis and implementation, the RISC-V GCC toolchain for firmware, SystemVerilog and Cocotb for verification, and simulation tools (iverilog/ModelSim as available).

3. Proposed System Design. The architecture (Fig. 1) couples the PicoRV32 core to a configurable accelerator over a lightweight memory and custom-instruction interface. The accelerator consists of a MAC array, a scratchpad for operand tiles, a controller/FSM, and a register interface for status. Custom instructions are decoded either through PicoRV32's PCPI port or a minimal extension of the decoder and execute stage.

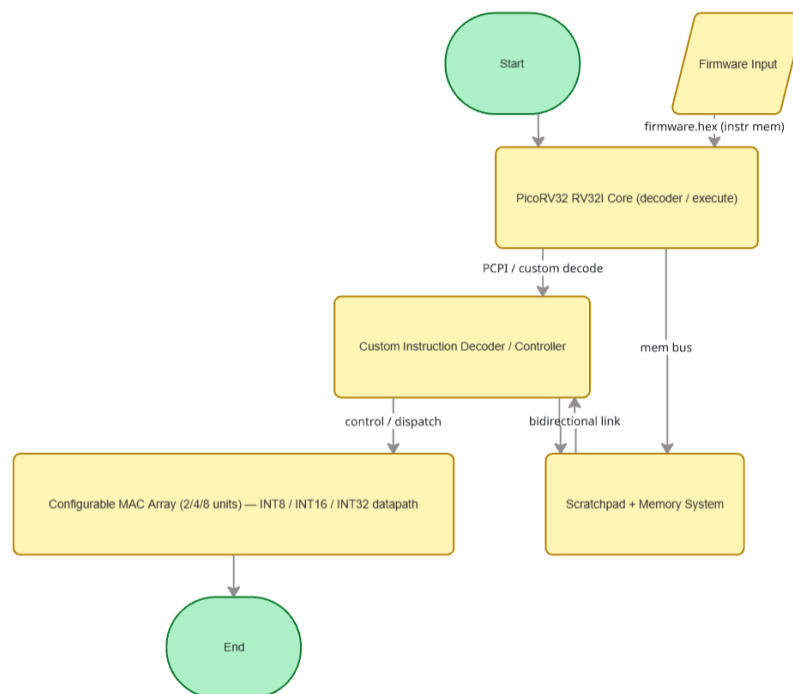


Fig. 1: Proposed architecture — configurable AI acceleration extension for an embedded RISC-V

processor on FPGA.

4. Implementation. The baseline PicoRV32 core was first brought up on the Basys 3 board, loading a small firmware image from instruction memory and verifying continuous execution via debug LEDs. The accelerator is then implemented in SystemVerilog with configurable MAC count and operand width, integrated with the core, and benchmark programs are cross-compiled with the RISC-V toolchain. Verification is performed with SystemVerilog testbenches and/or Cocotb, comparing hardware results against software references on randomized inputs.

5. Testing and Validation (Proposed). The system is tested with (a) unit tests for each custom instruction, (b) differential tests that compare accelerator outputs with golden software models, (c) regression and randomized tests for pipeline and handshake corner cases, and (d) on-board demonstration on the FPGA. Performance is measured in execution cycles, speedup, maximum clock frequency, LUT/BRAM/DSP utilization, and estimated power for each configuration.

6. Results and Analysis (Expected). Results will be tabulated and graphed to compare the 2/4/8-MAC configurations against the software baseline, and discussed against the stated objectives, including any limitations such as memory-bandwidth saturation or timing-closure constraints.

8.0 Expected Output

The expected outcomes of the project are:

A functional and reliable configurable AI acceleration extension integrated with the PicoRV32 embedded RISC-V processor on the Basys 3 FPGA, meeting all project requirements.

- A documented custom-instruction set (MATMUL, CONV, MAC, DOT, LOAD_TILE, STORE_TILE, WAIT) with a verified decoder and execution path.
- A benchmark suite (matrix multiplication, Sobel, Gaussian blur, FIR, dot product, tiny CNN) with software baselines and timing data.
- A verified RTL design (SystemVerilog) with a regression test suite, coverage report, and differential (hardware-vs-software) validation results.
- A synthesis and implementation flow producing bitstreams for the 2/4/8-MAC configurations, with timing, area, and power reports.
- A quantitative performance evaluation demonstrating multi-fold speedup (expected 5–20× on the 8-MAC configuration) over the software baseline, with explicit speedup–area–power trade-off graphs.
- A scalable and maintainable design whose MAC count and operand width can be re-parameterized, and which can be enhanced with additional kernels or a larger MAC array in future work.
- A complete research report and (in later phases) a research paper and thesis documenting the design, methodology, and results.

9.0 Other Relevant Information

9.1 Financial Arrangements

The budget is given below:

S/N	ITEM	DESCRIPTION	COST
1	FPGA Development Board	Digilent Basys 3 (Xilinx Artix-7 XC7A35T)	₹ 26,578
2	Cables and Accessories	USB programming cable, power supply, jumper wires, breadboard	₹ 1000
	Grand Total		₹ 27,578

Table 9.1: Budget of conducting project

9.2 Duration

This project will be completed in one year. The proposed schedule is given below:

SL.NO.	TASK NAME	2026					
		JUL	AUG	SEP	OCT	NOV	DEC
1	Literature review	✓					
2	Data collection & system analysis		✓	✓			
3	System Design and Development			✓	✓		
4	Prototype testing & installation				✓	✓	
5	Writing report					✓	✓
6	Submission						✓

Table 9.2: Proposed time schedule

10. References

- [1] A. Waterman and K. Asanović, Eds., *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, Version 20191213*. Berkeley, CA, USA: RISC-V Foundation, 2019. [Online]. Available: <https://riscv.org/specifications/>
- [2] C. Wolf, "PicoRV32: A Size-Optimized RISC-V CPU," GitHub, 2016. [Online]. Available: <https://github.com/YosysHQ/picorv32>
- [3] Y. Tortorella, L. Bertaccini, D. Rossi, L. Benini, and F. Conti, "RedMule: A Compact FP16 Matrix-Multiplication Accelerator for Adaptive Deep Learning on RISC-V-Based Ultra-Low-Power SoCs," in *Proc. IEEE/ACM Int. Symp. Low Power Electronics and Design (ISLPED)*, Boston, MA, USA, 2022, pp. 1–6.
- [4] M. Damian, J. Oppermann, C. Spang, and A. Koch, "SCAIE-V: An Open-Source SCALable Interface for ISA Extensions for RISC-V Processors," in *Proc. 59th ACM/IEEE Design Automation Conf. (DAC)*, San Francisco, CA, USA, 2022, pp. 1–6. doi: 10.1145/3489517.3530432.
- [5] E. Rezunov, N. Zurstraßen, L. M. Reimann, and R. Leupers, "Automatic Microarchitecture-Aware Custom Instruction Design for RISC-V Processors," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, Munich, Germany, 2025, pp. 1–9. doi: 10.1109/ICCAD66269.2025.11240781.
- [6] K. S. Gorde, S. M. Sonavane, S. Hutke, and A. Hutke, "Analog artificial intelligence hardware for neural networks: Design trends and considerations," *Bulletin of Electrical Engineering and Informatics*, vol. 14, no. 6, pp. 4399–4410, Dec. 2025. doi: 10.11591/eei.v14i6.10842.
- [7] N. P. Jouppi et al., "In-Datacenter Performance Analysis of a Tensor Processing Unit," in *Proc. 44th Annu. Int. Symp. Computer Architecture (ISCA)*, Toronto, Canada, 2017, pp. 1–12.
- [8] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.
- [9] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, "EIE: Efficient Inference Engine on Compressed Deep Neural Network," in *Proc. 43rd Annu. Int. Symp. Computer Architecture (ISCA)*, Seoul, South Korea, 2016, pp. 243–254.
- [10] M. R. Guthaus, J. S. Ringenberg, D. Ernst, T. M. Austin, T. Mudge, and R. B. Brown, "MiBench: A free, commercially representative embedded benchmark suite," in *Proc. IEEE Int. Workshop on Workload Characterization (WWC-4)*, Austin, TX, USA, 2001, pp. 3–14.
- [11] Xilinx Inc., "7 Series FPGAs Data Sheet: Overview," DS180, v2.6.1, San Jose, CA, USA, 2018.
- [12] F. Conti, D. Rossi, A. Pullini, I. Loi, and L. Benini, "PULP: A Ultra-Low Power Parallel Accelerator for Energy-Efficient and Flexible Embedded Vision," *Journal of Signal Processing Systems*, vol. 84, no. 3, pp. 339–354, 2016.

CANDIDATES

Name: MD Sameer Ahmed Reg. No 2310040004

Signature: Date: August 3, 2026

Name: Sayooj S Reg. No. 2310040095

Signature: Date: August 3, 2026

SUPERVISOR

1. Comments by Supervisor:

.....
.....
.....
.....

Date: Name:

Signature:

2. Comments by Supervisor:

.....
.....
.....
.....

Date: Name:

Signature: